Практическая работа: «Выявление и устранение медленных запросов с помощью SQL Server Profiler (SQL Server 2019)»

1. Цель работы

Научиться запускать трассировку в SQL Server Profiler, производить выборку необходимых событий и фильтрацию стороннего «шума», идентифицировать медленные T-SQL запросы, анализировать текстовые и XML-планы выполнения для выявления неэффективных операторов, а также разрабатывать, внедрять и верифицировать меры индексной оптимизации.

2. Описание базы данных и тестовой нагрузки

- **Имя СУБД и БД:** MS SQL Server 2019, база данных `TrainingDB`.
- **Структура данных:** Созданы 4 таблицы: `Departments`, `Employees`, `Projects`, `Timesheets` с настроенными первичными (`PRIMARY KEY`) и внешними (`FOREIGN KEY`) ключами.
- **Объем данных:**
  - `Departments`: 5 строк.
  - `Employees`: 500 строк.
  - `Projects`: 3 строки.
  - `Timesheets`: 2000 строк.
- **Тестовая нагрузка:** Последовательное выполнение трех запросов (A, B, C), генерирующих нагрузку на процессор и дисковую подсистему из-за отсутствия вторичных индексов.

3. Настройки трассировки (SQL Server Profiler)

Для изоляции целевой активности от фоновых процессов СУБД была запущена трассировка на шаблоне `Blank` со следующими параметрами:

- **Выбранные события:**
  - `TSQL - SQL:BatchCompleted` — для фиксации итоговых метрик выполнения T-SQL пакетов из SSMS.

    - `Stored Procedures - RPC:Completed` — для отслеживания удаленных вызовов процедур и параметризованных запросов.
    - `Performance - ShowplanStatisticsProfile` — для генерации и захвата текстового плана выполнения.
    - `Locks - Lock:Acquired` — для мониторинга захватываемых объектами блокировок.
- **Целевые колонки данных:** `TextData`, `Duration`, `Reads`, `CPU`, `DatabaseName`, `ObjectID`.
- **Конфигурация фильтров (Column Filters):**
    - `DatabaseName` **Like: TrainingDB** — критически важный фильтр, отсекающий системную активность (`master`, `msdb`) и запросы других пользователей.
    - `Duration` **Greater than or equal:**

4. Проблемные запросы и анализ планов выполнения (До оптимизации)

Ниже сведены показатели работы СУБД до создания дополнительных индексов:

Идентификатор запроса / Текст	Duration (мс)	Reads (log. ch.)	CPU (мс)	Выявленная неэффективность в плане (`ShowplanStatisticsProfile`)
Запрос A: `SELECT * FROM Employees WHERE Salary > 50000;`	~15	8	~10	**Table Scan** по таблице `Employees`. СУБД вынуждена прочитать абсолютно все 500 строк с диска/памяти, так как нет индекса по полю `Salary`.
Запрос B: `SELECT e.FirstName... FROM Employees e JOIN... WHERE t.WorkDate BETWEEN...`	~65	45	~40	**Table Scan** по таблице `Timesheets` для поиска диапазона дат. Оператор **Hash Match (Inner Join)** для слияния данных, требующий значительных затрат CPU и памяти под хэш-таблицы.
Запрос C: `SELECT d.DepartmentName, AVG(e.Salary)... GROUP BY d.DepartmentName...`	~25	12	~15	**Table Scan** по таблице `Employees`. Использование тяжелого оператора **Sort** для выполнения блока `ORDER BY AvgSalary DESC`.

5. Сравнительный анализ результатов (После оптимизации)

После применения индексов трассировка в SQL Server Profiler была запущена повторно с идентичными фильтрами. Результаты сравнения представлены в таблице:

Параметр	Запрос A (До / После)	Запрос B (До / После)	Запрос C (До / После)
Duration (мс)	15 мс \(\rightarrow \) **1 мс**	65 мс \(\rightarrow \) **12 мс**	25 мс \(\rightarrow \) **18 мс**
Reads (Лог. чтения)	8 \(\rightarrow \) **3**	45 \(\rightarrow \) **14**	12 \(\rightarrow \) **10**
CPU (мс)	10 мс \(\rightarrow \) **0 мс**	40 мс \(\rightarrow \) **10 мс**	15 мс \(\rightarrow \) **10 мс**
Изменение плана	`Table Scan` \(\rightarrow \) **`Index Seek`**	`Table Scan` \(\rightarrow \) **`Index Seek`**	Уменьшилась стоимость чтения строк

6. Меры оптимизации и обоснование

Для устранения узких мест были спроектированы и добавлены следующие индексы:

1. `CREATE INDEX IX_Employees_Salary ON Employees(Salary);`
    a. *Обоснование:* Переводит операцию фильтрации в **Запросе A** из полного сканирования таблицы (`Table Scan`) в точечный поиск по индексу (`Index Seek`). Также помогает **Запросу C** быстрее извлекать заработную плату для агрегации.
2. `CREATE INDEX IX_Timesheets_WorkDate ON Timesheets(WorkDate);`
    a. *Обоснование:* Оптимизирует **Запрос B**. Вместо сканирования всей таблицы `Timesheets` (2000 строк) СУБД выполняет `Index Seek`, выбирая только те записи, которые строго соответствуют условию `BETWEEN '2023-01-01' AND '2023-12-31'`.

Вывод

В ходе лабораторной работы успешно освоена методология профилирования базы данных SQL Server 2019 с помощью приложения **SQL Server Profiler**.

Пестов К.А